

渡船 (ferry)

【题目描述】

海上散布着 n 座岛屿，岛屿之间共有 m 条双向航线。第 i ($1 \leq i \leq m$) 条航线连接岛屿 u_i 与 v_i ，并且在这条航线上有一艘渡船往返运行。

每艘渡船有一个初始停泊位置。对于第 i ($1 \leq i \leq m$) 条航线上的渡船，

- 若 $w_i = 0$ ，则渡船初始停泊在岛屿 u_i ；
- 若 $w_i = 1$ ，则你可以任意决定渡船初始停泊在岛屿 u_i 或岛屿 v_i 。

现在有 k 位旅人，第 j ($1 \leq j \leq k$) 位旅人初始位于岛屿 s_j ，需要前往岛屿 t_j 。旅人只能通过渡船移动。具体地，每次移动会选择一位旅人 j ($1 \leq j \leq k$) 与一条航线 i ($1 \leq i \leq m$)，满足旅人 j 与航线 i 上的渡船当前位于同一座岛屿，然后旅人将会登上渡船，与渡船一同移动到航线 i 另一端的岛屿。例如，若旅人 j 与航线 i 上的渡船均位于岛屿 u_i ，则移动后旅人与渡船均会抵达岛屿 v_i ，反之亦然。

你需要判断，是否可以通过一系列旅人的移动，使得所有旅人均能到达各自的目的地。若存在，你还需要给出一组具体的移动方案。

【实现细节】

选手不需要，也不应该实现 `main` 函数。

选手需要确保提交的程序包含头文件 `ferry.h`，即在程序开头加入以下代码：

```
1 #include "ferry.h"
```

选手需要在提交的程序源文件中实现以下函数：

```
1 void ferry(int n, int m, int k, std::vector<int> u,
    std::vector<int> v, std::vector<int> w, std::vector<int> s,
    std::vector<int> t);
```

- n, m, k 分别表示岛屿的数量、航线的数量与旅人的数量。
- 对于 $0 \leq i < m$ ， u_i, v_i 分别表示第 $i + 1$ 条航线连接的两座岛屿， w_i 表示该航线上的渡船的初始停泊位置是否确定，具体见【题目描述】。
- 对于 $0 \leq j < k$ ， s_j, t_j 分别表示第 $j + 1$ 位旅人初始位于的岛屿与目的地岛屿。
- 对于每个测试点，该函数会被交互库调用恰好一次。

选手可以通过调用以下函数报告是否可以通过一系列旅人的移动，使得所有旅人均能到达各自的目的地：

```
1 void report(bool o);
```

- 若 o 为 `true`，则表示可以通过一系列旅人的移动，使得所有旅人均能到达各自的目的地；若 o 为 `false`，则表示不可以。

- 选手需要保证交互库调用 `ferry` 时，恰好调用了一次该函数。

选手可以通过调用以下函数进行一次移动：

```
1 void move(int j, int i);
```

- j, i 分别表示选择的旅人与航线的编号，具体含义如【题目描述】中所示。选手需要保证 $1 \leq j \leq k$, $1 \leq i \leq m$ ，且旅人 j 与航线 i 上的渡船当前位于同一座岛屿。特别地，若航线 i 上的渡船的初始停泊位置还未确定，且旅人 j 位于航线 i 连接的其中一座岛屿，则航线 i 上的渡船的初始停泊位置直接被确定为旅人 j 所在的岛屿。
- 选手需要保证交互库每次调用 `ferry` 时，调用该函数的次数不超过 10^6 ，且所有该函数的调用均在调用 `report` 函数之后，且 `report` 函数的返回值必须为 `true`。

注意：在任何情况下，交互库运行所需时间均不会超过 0.1 秒，所用内存为固定大小，且均不超过 64 MiB。

【测试程序方式】

试题目录下的 `grader.cpp` 是交互库参考实现，最终测试时所用的交互库实现与该参考实现有所不同，因此选手的解法不应该依赖交互库实现。

选手可以在本题目录下使用如下命令编译得到可执行程序：

```
1 g++ grader.cpp ferry.cpp -o ferry -std=gnu++14 -O2 -pipe -static -s
```

对于编译得到的可执行程序：

- 可执行文件将从标准输入读入以下格式的数据：
 - 输入的第一行包含三个正整数 n, m, k ，分别表示岛屿的数量、航线的数量与旅人的数量。
 - 输入的第 $i + 1$ ($1 \leq i \leq m$) 行包含三个非负整数 u_i, v_i, w_i ，分别表示第 i 条航线连接的两座岛屿与该航线上的渡船的初始停泊位置是否确定，具体见【题目描述】。
 - 输入的第 $n + 2$ 行包含 k 个正整数 $s_1, \dots, s_k$ ，分别表示每位旅人初始位于的岛屿。
 - 输入的第 $n + 3$ 行包含 k 个正整数 $t_1, \dots, t_k$ ，分别表示每位旅人的目的地岛屿。
- 可执行文件将输出以下格式的数据至标准输出：
 - 输出的第一行包含一个字符串 `Yes` 或 `No`，表示是否可以通过一系列旅人的移动，使得所有旅人均能到达各自的目的地。
 - 若可以，则

- * 输出的第二行包含一个非负整数 c ，表示移动次数；
- * 输出的第 $l+2$ ($1 \leq i \leq c$) 行包含两个正整数 j, i ，分别表示第 l 次移动选择的旅人与航线的编号。

【样例 1 输入】

```
1 4 5 2
2 1 2 1
3 3 1 0
4 3 2 0
5 2 4 1
6 4 3 0
7 1 4
8 3 2
```

【样例 1 输出】

```
1 Yes
2 4
3 2 5
4 2 3
5 1 1
6 1 3
```

【样例 1 解释】

- 初始时，旅人 1 位于岛屿 1，旅人 2 位于岛屿 4；
- 第一次移动后，旅人 2 与航线 5 上的渡船一同移动到岛屿 3；
- 第二次移动后，旅人 2 与航线 3 上的渡船一同移动到岛屿 2；
- 第三次移动时，航线 1 上的渡船的初始停泊位置被确定为岛屿 1，移动后旅人 1 与航线 1 上的渡船一同移动到岛屿 2；
- 第四次移动后，旅人 1 与航线 3 上的渡船一同移动到岛屿 3。

【附加文件说明】

在附加文件中：

1. `grader.cpp` 是提供的交互库参考实现。
2. `ferry.h` 是头文件，选手不用关心具体内容。

3. `template_ferry.cpp` 是提供的示例代码，选手可参考并实现自己的代码。

【子任务】

对于所有测试数据，均有：

- $2 \leq n \leq 2,000$, $1 \leq m \leq 10^4$, $1 \leq k \leq 10^2$;
- 对于所有 $1 \leq i \leq m$ ，均有 $1 \leq u_i, v_i \leq n$, $u_i \neq v_i$, $w_i \in \{0, 1\}$;
- 对于所有 $1 \leq i \leq k$ ，均有 $1 \leq s_i, t_i \leq n$ 。

子任务编号	分值	特殊性质
1	6	$k = 1$
2	7	$k = n$ ，且 $s_1, \dots, s_n$ 与 $t_1, \dots, t_n$ 均为 $1 \sim n$ 的排列
3	11	对于所有 $1 \leq i \leq m$ ，均有 $w_i = 0$
4	9	对于所有 $1 \leq i \leq m$ ，均有 $w_i = 1$
5	29	存在一个 $1 \sim k$ 的排列 p ，满足对于所有 $1 \leq i \leq k$ ，均有 $s_i = t_{p_i}$
6	26	$n \leq 100$, $m \leq 200$, $k \leq 60$
7	12	无

【评分方式】

注意：

- 选手不应当通过非法方式获取交互库的内部信息，如直接与标准输入、输出流进行交互。此类行为将被视为作弊；
- 最终的评测交互库与样例交互库的实现不同。

本题首先会受到和传统题相同的限制，例如编译错误会导致整道题目得 0 分，运行时错误、超过时间限制、超过空间限制等会导致相应测试点得 0 分等。选手只能在程序中访问自己定义的变量以及交互库给出的变量，尝试访问其他地址空间将可能导致编译错误或运行错误。

每次调用 `ferry` 函数时，若 `report` 函数或 `move` 函数调用不合法，或 `move` 函数调用次数超过 10^6 ，则相应测试点得 0 分。

在上述条件基础上：

- 对于每个测试点，若 `report` 函数返回值正确，则可以获得 40% 的分数；在此基础上，若 `report` 返回值为 `false`，或 `ferry` 函数返回时所有旅人均到达对应的目的地岛屿，则可以获得满分。

Problem J. Permutation

Input file: *standard input*
Output file: *standard output*
Time limit: 1 second
Memory limit: 256 mebibytes

You are given a permutation $p_1, p_2, \dots, p_n$. You can do the following operations repeatedly:

- Choose an interval $p_l, p_{l+1}, \dots, p_{l+c}$ ($l \geq 1, l+c \leq n$) where p_l is the smallest element in this interval, you can permute $p_{l+1}, \dots, p_{l+c}$ in arbitrary way.
- Choose an interval $p_l, p_{l+1}, \dots, p_{l+c}$ ($l \geq 1, l+c \leq n$) where p_{l+c} is the smallest element in this interval, you can permute $p_l, \dots, p_{l+c-1}$ in arbitrary way.

You want to know how many distinct permutations you can get using operations. The answer can be large, output the answer modulo 998244353.

Input

The first line contains an integer T denoting the number of test cases ($1 \leq T \leq 100000$).

The first line in a test case contains two integers n and c ($2 \leq c \leq 500000, 2 \leq n \leq 500000$). The sum of n over all test cases does not exceed 500000.

The second line in a test case contains a permutation $p_1, \dots, p_n$ ($1 \leq p_i \leq n$).

Output

For each test case, output one line containing the answer modulo 998244353.

Example

standard input	standard output
5	6
5 3	1
3 4 2 1 5	4
5 4	6
4 2 1 3 5	4
5 2	
4 5 3 1 2	
5 3	
4 3 2 1 5	
5 2	
2 3 1 5 4	

Problem C. 广义串并联图计数

广义串并联图计数

- 时间限制：2 秒
- 空间限制：512 MB

题目描述

定义一张 **有标号** 无向图是简单广义串并联图，当且仅当：

1. 图联通；
2. 图上 **不存在** 重边和自环；
3. 对于任意 4 个节点都 **不存在** 6 条两两没有公共点的路径连接这 4 个节点中的每一对节点。

定义 $f(i, j)$ 表示有多少张 i 个点 j 条边的简单广义串并联图。

给定 n, m, p ，你要求出所有 $1 \leq i \leq n, 1 \leq j \leq m$ 的 $f(i, j) \bmod p$ 的结果。

输入格式

共一行，包含三个正整数 n, m, p 。

输出格式

输出 n 行，每行包含 m 个整数，其中第 i 行的第 j 个整数表示 $f(i, j) \bmod p$ 的结果。

样例 1 输入

5 10 998244353

样例 1 输出

```
0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0
0 3 1 0 0 0 0 0 0 0
0 0 16 15 6 0 0 0 0 0
0 0 0 125 222 205 70 0 0 0
```

样例 2

见附加文件。

数据范围

对于所有数据，保证：

- $1 \leq n, m \leq 50$;
- $10^8 \leq p \leq 10^9 + 7$, p 是质数。

本题采用捆绑测试。

- Subtask 1 (8 points) : $n \leq 8$, $m \leq 10$ 。
- Subtask 2 (12 points) : $n \leq 10$, $m \leq 20$ 。
- Subtask 3 (12 points) : $n \leq 11$, $m \leq 20$ 。
- Subtask 4 (32 points) : $n, m \leq 25$ 。
- Subtask 5 (36 points) : 无特殊限制。

Problem D. Secret Lilies and Roses

AI Translation — This statement was translated using AI and may contain inaccuracies. Please refer to the original statement if in doubt. 

有 n 朵花从左到右排成一行，依次编号为 1 到 n 。每朵花要么是百合，要么是玫瑰。对于 0 到 n 之间的整数 j （包含 0 和 n ），令 l_j 表示最左侧 j 朵花中百合的数量，令 r_j 表示最右侧 $n - j$ 朵花中玫瑰的数量。

最初，你只知道花的数量 n 。花的种类是隐藏的。你可以通过询问来获取有关花的信息。每次询问可以执行以下操作之一：

类型询问 (Type query)：指定一个 1 到 n 之间的整数 i 。你将收到第 i 朵花的种类。

乘法询问 (Multiply query)：指定一个 0 到 n 之间的整数 j 。你将收到 $l_j \times r_j$ 的值。

你的任务是通过有限次数的询问，找到一个 0 到 n 之间的整数 k ，使得 $l_k = r_k$ 。你可以假设对于给定的花卉排列，至少存在一个这样的整数。注意，你不需要确定每一朵花的种类。

交互

输入的第一行包含一个整数 t ($1 \leq t \leq 100$)，表示测试用例的数量。随后是 t 个测试用例。每个测试用例的格式如下：

每个测试用例的第一行包含一个整数 n ($1 \leq n \leq 100$)。读取该值后，你的程序可以开始进行询问。对于每次询问，你的程序应输出一行，包含以下内容之一：

- “type i ”：执行类型询问，指定整数 i ($1 \leq i \leq n$)。作为响应，你将从输入中读取一行字符串 (lily 或 rose)，表示第 i 朵花的种类。
- “multi j ”：执行乘法询问，指定整数 j ($0 \leq j \leq n$)。作为响应，你将从输入中读取一行整数，表示 $l_j \times r_j$ 的值。

每个测试用例允许进行最多 10 次询问。这意味着类型询问和乘法询问的总次数不得超过 10 次。如果你的程序询问次数超过 10 次，将被判为“Wrong Answer”。

当你的程序确定了一个满足 $l_k = r_k$ 的整数 k 时，应输出一行格式为“answer k ”的内容 ($0 \leq k \leq n$)。注意，输出答案不计入询问次数。

题目保证至少存在一个正确的输出。如果存在多个正确的输出，接受其中任何一个即可。

输出答案行后，你的程序应开始处理下一个测试用例。当所有 t 个测试用例处理完毕后，交互结束，你的程序应终止。

关于交互式评测的说明：

- 评测是非对抗性的，这意味着花的种类是预先选定的，而不是根据你的询问实时生成的。
- 输出后请务必刷新缓冲区。详情请参阅“评测细节”文档。
- 你将获得一个用于本地测试的命令行工具，以及与样例交互对应的输入文件。你可以从 DOMjudge 下载这些文件。工具顶部有关于其用法的注释。

样例

输入 1

```
2
9
lily
rose
6
3
3
0
rose
```

输出 1

```
type 8
type 1
multi 6
multi 3
answer 5
multi 3
type 3
answer 3
```

说明 1

对于第一个测试用例，九朵花的排列如下图所示。响应第三次询问时，返回 $l_6 \times r_6 = 3 \times 2 = 6$ ；响应第四次询问时，返回 $l_3 \times r_3 = 1 \times 3 = 3$ 。由于 $l_5 = r_5 = 3$ ，因此 $k = 5$ 是一个正确的输出。



对于第二个测试用例，假设所有三朵花都是玫瑰。

Problem E. Find the Tree

Time Limit	2000 ms
Mem Limit	262144 kB
Code Length Limit	10240 B

You should find the edges of a tree with N nodes. You can ask queries of the type:

- For a chosen triplet of nodes (A, B, C) , which is the node D that minimizes the sum of distances $AD + BD + CD$?

The tree is generated using the following algorithm:

- Initially we consider the tree consisting of a single node labeled 1
- Then we add the nodes one by one, in increasing order of labels from 2 to N
- When we add the node labeled i we connect it to one of the previously added nodes randomly (using a uniform distribution).
- After we finish generating the tree we relabel the nodes applying a random permutation.

Interaction

First you should read a single integer N .

Then you can start asking the queries. Each query should consist of the character **Q** followed by three integers A , B and C .

When you determine the tree, print the character **A** followed by $N - 1$ pairs of integers representing the edges.

Constraints and notes

- This task is *NOT* **adaptive task** value="adaptive"
- $2 \leq N \leq 2000$
- You can ask at most 25 000 queries

Example 1

Input	Output
7 1 2 2 3 3 4 4 5 4 6 3 7	

Problem F. Balanced String

Time Limit	500 ms
Mem Limit	131072 kB
Code Length Limit	10240 B

We define a /circular substring/ of a string S to be a substring that can also cross over the end of the string and continue from the beginning. For example, the string **EFAB** is considered a circular substring of length 4 of the string **ABCDEF**.

A string S is considered to be a /balanced string/ if the following conditions hold true:

- Each character in the string is either **A** or **B**.
- For any two circular substrings of S of the same length l ($1 \leq l \leq N$) the number of **A**s in the first substring and the number of **A**s in the second substring differ by at most 1.

Given T different strings that already satisfy the first condition, print for each of them whether they satisfy the second condition as well.

Standard input

The first line contains an integer value, T .

Each of the next T lines contains a string containing only **A** and **B**.

Standard output

Output T lines, each representing the answer for a string. If the /balanced string/ conditions are met output 1, otherwise output 0

Constraints and notes

- $1 \leq T \leq 20$
- $1 \leq |S| \leq 50.000$

Example 1

Input	Output
5	0
ABAABAABAB	1
AABABAABAB	0
ABBABBA	0
AABB	1
ABABABA	

Problem G. High Elements

Time Limit 2000 ms

题目描述

给定一个长度为 $(1, 2, \dots, N)$ 的排列 P 。

由字符 **0** 和 **1** 组成的长度为 N 的字符串 S 被称为**优质字符串**，当且仅当满足以下条件：

- 序列 X 和 Y 按如下方式构建：
 - 初始时， X 和 Y 为空序列
 - 对于每个 $i = 1, 2, \dots, N$ ，按顺序判断：若 $S_i = \mathbf{0}$ ，则将 P_i 追加到 X 末尾；若 $S_i = \mathbf{1}$ ，则追加到 Y 末尾
- 当 X 和 Y 的**高峰元素**数量相同时， S 即为优质字符串。其中，序列中第 i 个元素被称为**高峰元素**，当且仅当该元素是从第1到第 i 个元素中的最大值。

请判断是否存在优质字符串。若存在，找出字典序最小的那个。

约束条件

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq P_i \leq N$
- $P_1, P_2, \dots, P_N$ 互不相同
- 输入均为整数

输入格式

从标准输入按以下格式获取数据：

```
N
P1 P2 ... PN
```

输出格式

若不存在优质字符串，输出 **-1**。若存在，输出字典序最小的优质字符串。

样例1

Input	Output
6 3 1 4 6 2 5	001001

设 $S = 001001$ 。此时 $X = (3, 1, 6, 2)$, $Y = (4, 5)$ 。在序列 X 中，第一和第三个元素是高峰元素；在 Y 中，第一和第二个元素是高峰元素。由于两者高峰元素数量相同， 001001 即为优质字符串。不存在字典序更小的优质字符串，因此答案为 001001 。

样例2

Input	Output
5 1 2 3 4 5	-1

样例3

Input	Output
7 1 3 2 5 6 4 7	0001101

样例4

Input	Output
30 1 2 6 3 5 7 9 8 11 12 10 13 16 23 15 18 14 24 22 26 19 21 28 17 4 27 29 25 20 30	0000000000001100101010010011101

Problem H. Lost Permutation

Time Limit 2000 ms

Mem Limit 524288 kB

OS Windows

Statement [Statements \(PDF\)_\(en\)](#)

This is an interactive problem.

You once had a permutation π of size n . And now it's gone. All you have left is an old device you made while studying group theory. To try and recover π you can input a permutation f of size n into this device. This device will then display a permutation $\pi^{-1} \circ f \circ \pi$. Find π using at most two interactions with the device.

A permutation of size n is a sequence of n distinct integers from 1 to n . The *composition* of two permutations a and b is a permutation $a \circ b$ such that $(a \circ b)_i = b_{a_i}$. That is, if we consider a permutation as an action on n elements, moving element at position i to a_i , then $a \circ b$ is the action that applies a , then applies b , so that element at position i first moves to a_i , then moves to b_{a_i} . Note that some definitions of composition use the reverse order.

The inverse permutation π^{-1} is a permutation σ such that $\sigma_{\pi_i} = i$. The composition of a permutation and its inverse is equal to an identity permutation: $(\pi \circ \pi^{-1})_i = (\pi^{-1} \circ \pi)_i = i$ for all i from 1 to n . For example, if $a = (4, 1, 3, 2)$ and $b = (3, 2, 1, 4)$, then $a \circ b = (4, 3, 1, 2)$, $a^{-1} = (2, 4, 3, 1)$ and $a^{-1} \circ b \circ a = (1, 2, 4, 3)$.

Interaction

Your program has to process multiple test cases in a single run. First, the testing system writes t , the number of test cases ($t \geq 1$). Then, t test cases should be processed one by one.

In each test case your program should start by reading the integer n ($3 \leq n \leq 10^4$), the size of permutation π . The sum of n over all test cases does not exceed 10^4 . Then, your program can make queries of two types:

- ? $f_1 f_2 \dots f_n$, values $f_1, f_2, \dots, f_n$ form a permutation of $1, 2, \dots, n$. The testing system responds with a permutation $g_1, g_2, \dots, g_n$, where $g = \pi^{-1} \circ f \circ \pi$.
- ! $\pi_1 \pi_2 \dots \pi_n$ — your guess for the secret permutation.

You can use at most two queries of the first type in each test case. After your program makes a query of the second type, it should continue to the next test case (or exit if that test case was the last one).

Examples

Input	Output
2 4	? 3 2 1 4
1 2 4 3	? 2 4 3 1
2 4 3 1	! 4 1 3 2
3	? 2 3 1
3 1 2	? 3 1 2
2 3 1	! 3 2 1

Note

There are two test cases in the first test. In the first test case, $\pi = (4, 1, 3, 2)$ is the only permutation that satisfies $\pi^{-1} \circ (3, 2, 1, 4) \circ \pi = (1, 2, 4, 3)$ and $\pi^{-1} \circ (2, 4, 3, 1) \circ \pi = (2, 4, 3, 1)$. In the second test case, based on the interaction, π can be equal to either $(1, 3, 2)$, $(2, 1, 3)$, or $(3, 2, 1)$. The solution got lucky and guessed the correct one: $(3, 2, 1)$.